

ЗВІТ

до лабораторної роботи № 4

на тему:

"Розширення можливостей PostgreSQL: користувацькі типи, функції та тригери."

Виконала:

студентка групи МІТ-31

Йовхимиш Діана

Мета роботи: закріпити знання з розширюваності PostgreSQL. Навчитися створювати користувацькі типи даних. Реалізувати власну користувацьку функцію або агрегат. Створити тригери для логування змін у базі даних. Автоматично оновлювати пов'язані таблиці чи заповнювати значення. Оновити діаграму бази даних відповідно до виконаних завдань. Перевірити коректність роботи реалізованих об'єктів через виконання тестових SQL-запитів.

Хід роботи

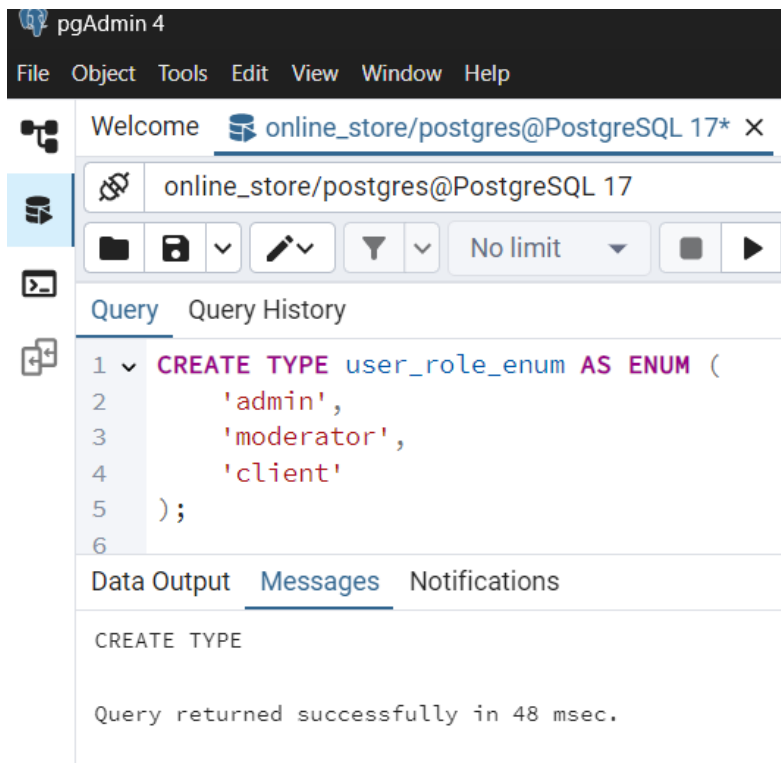
Завдання:

1. Створення користувацького типу даних

1. Визначити один або кілька атрибутів у базі даних, які можна винести в окремий користувацький тип даних. Це може бути ENUM (для обмеженого набору значень) або Composite Type (якщо потрібно зберігати структуровані дані в одному полі).

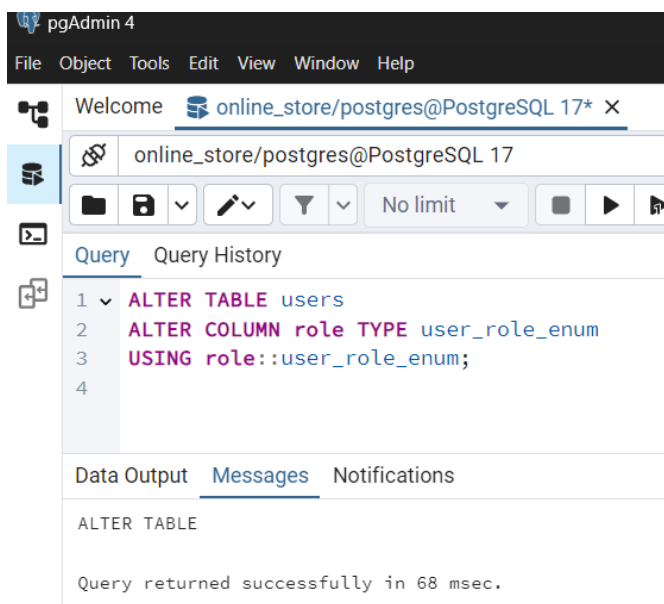
На мою думку, **атрибут role** у моїй **таблиці users** доцільно винести в користувацький тип даних **ENUM**, оскільки він має обмежений і фіксований набір допустимих значень (admin, moderator, client). Використання **ENUM** дозволить виключити некоректні значення, підвищити цілісність даних та забезпечити можливість розширення системи шляхом додавання нових ролей.

2. Створити користувацький тип, що відповідає логіці інформаційної системи та забезпечує розширюваність бази даних:



Створення користувацького ENUM-типу.

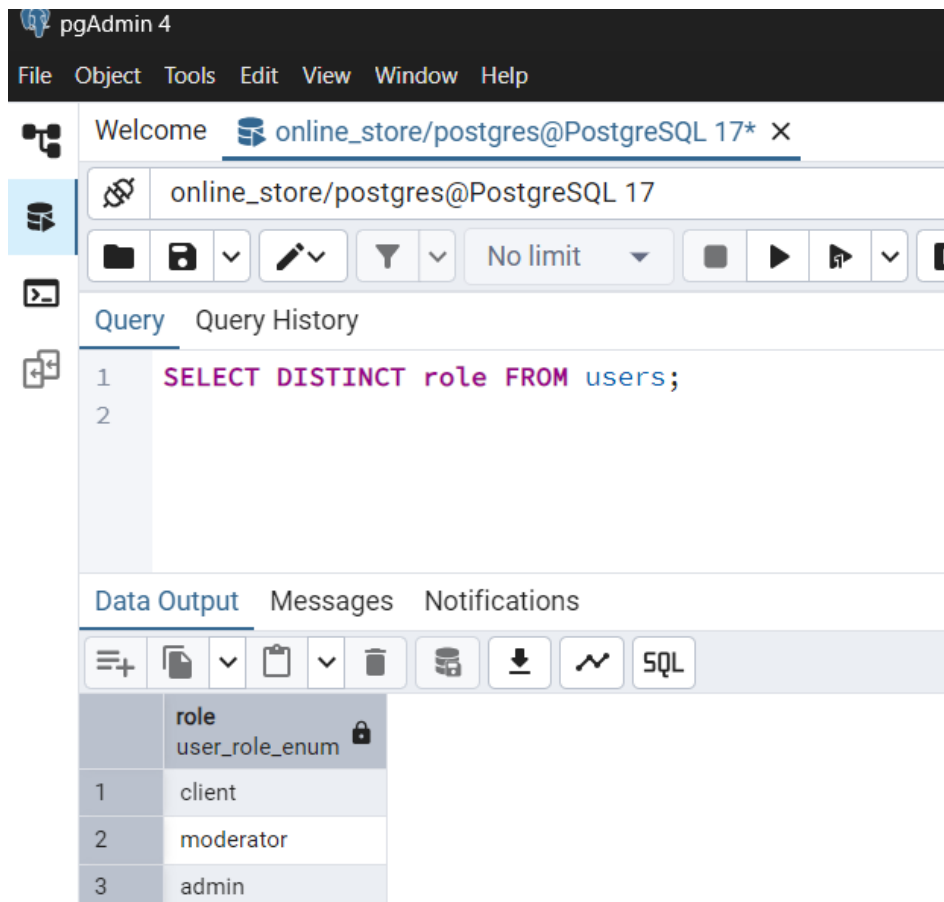
3. Додати новий стовпець із цим типом до відповідної таблиці в базі даних, змінюючи її структуру для кращої адаптації до майбутніх оновлень:



Зміна структури таблиці users.

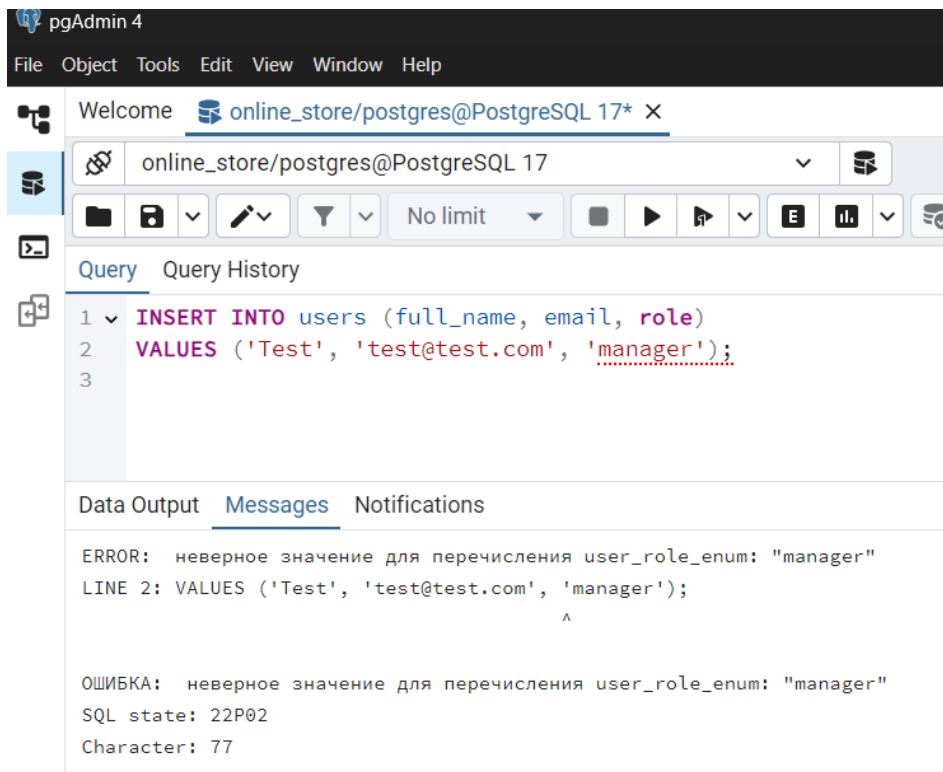
4. Переконалися, що зміни не порушують цілісність даних, підтримують взаємозв'язки між таблицями та узгоджуються з існуючою структурою бази:

Перевірка допустимих значень:



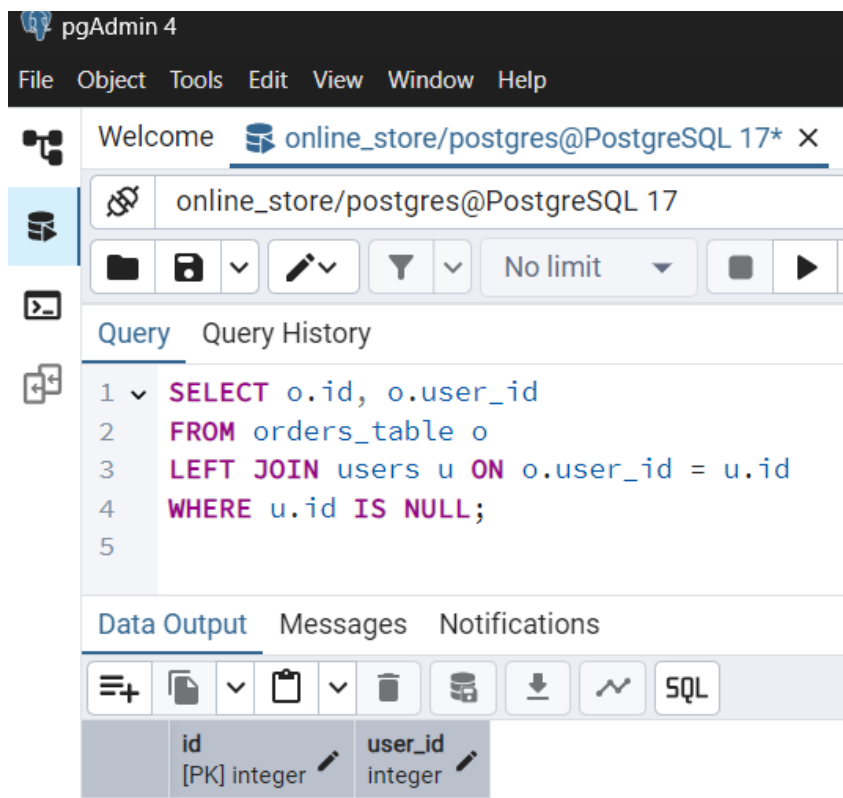
Усі значення належать допустимому домену.

Перевірка захисту від некоректних значень:



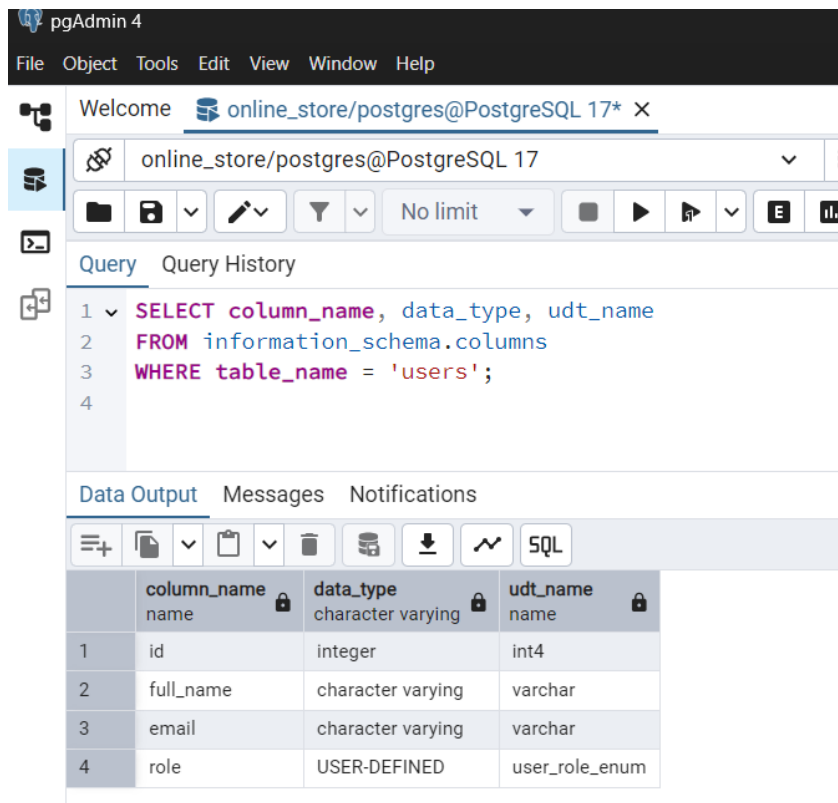
База даних блокує некоректні дані - не порушується цілісність.

Перевірка, що зв'язки між таблицями не порушені:



Усі замовлення прив'язані до існуючих користувачів.

Перевірка структури таблиці users:



Усі стовпці збережені без змін, атрибут role успішно приведено до користувацького типу `user_role_enum`, а типи інших полів відповідають початковій структурі таблиці.

2. Створення користувацької функції або агрегату

1. Реалізувати функцію або агрегат, що підходить для аналізу даних вашої інформаційної системи.
2. Функція повинна виконувати розширені обчислення або маніпуляції з даними, які неможливо здійснити стандартними SQL-запитами.
3. Використати створену функцію/агрегат у тестовому запиті до таблиць бази даних та перевірити її ефективність.
4. Дослідити, як використання таких функцій покращує продуктивність та зручність роботи з базою даних.

Створила функцію, яка підраховує загальну суму замовлень для конкретного користувача та середній чек:

Welcome online_store/postgres@PostgreSQL 17* X

online_store/postgres@PostgreSQL 17

Query Query History

```
1 CREATE OR REPLACE FUNCTION get_user_order_stats(p_user_id INT)
2 RETURNS TABLE(
3     total_orders INT,
4     total_spent DECIMAL,
5     avg_order_amount DECIMAL
6 ) AS $$
7 BEGIN
8     RETURN QUERY
9     SELECT
10         COUNT(o.id)::INT AS total_orders,
11         COALESCE(SUM(p.amount), 0) AS total_spent,
12         COALESCE(AVG(p.amount), 0) AS avg_order_amount
13     FROM orders_table o
14     LEFT JOIN payments p ON p.order_id = o.id
15     WHERE o.user_id = p_user_id;
16 END;
17 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 116 msec.

Тестування функції для одного користувача:

Welcome online_store/postgres@PostgreSQL 17* x

online_store/postgres@PostgreSQL 17

Query Query History

```
1 SELECT * FROM get_user_order_stats(1);
```

Data Output Messages Notifications

	total_orders integer	total_spent numeric	avg_order_amount numeric
1	2	77100.00	38550.000000000000

Статистика для всіх користувачів:

Welcome online_store/postgres@PostgreSQL 17* x

online_store/postgres@PostgreSQL 17

Query Query History

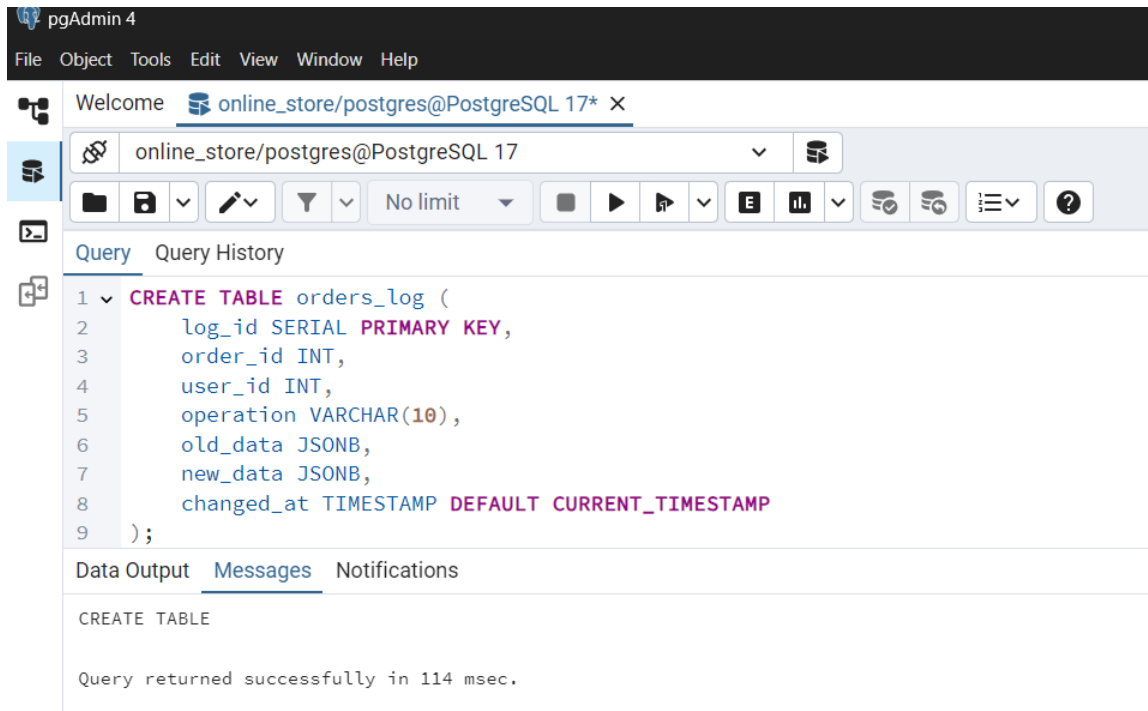
```
1 SELECT
2     u.full_name,
3     stats.total_orders,
4     stats.total_spent,
5     stats.avg_order_amount
6 FROM users u
7 CROSS JOIN LATERAL get_user_order_stats(u.id) AS stats
8 ORDER BY stats.total_spent DESC;
```

Data Output Messages Notifications

	full_name character varying (100)	total_orders integer	total_spent numeric	avg_order_amount numeric
1	Йовхимищ Діана	2	77100.00	38550.000000000000
2	Власюк Лариса	1	30000.00	30000.000000000000
3	Павленко Анна	1	20000.00	20000.000000000000
4	Ткаченко Олена	1	5450.00	5450.00000000000000

3. Створення тригерів для логування змін та автоматичного оновлення пов'язаних таблиць

1. Створити таблицю логування змін у критично важливих таблицях вашої бази даних:



2. Написати тригерну функцію на PL/pgSQL для логування операцій INSERT, UPDATE та DELETE, забезпечуючи можливість відстеження змін у базі:

The screenshot shows the pgAdmin 4 interface. The top bar includes the menu (File, Object, Tools, Edit, View, Window, Help) and the connection name 'online_store/postgres@PostgreSQL 17'. Below the toolbar, the 'Query' tab is active, displaying a SQL script to create a trigger function. The script is as follows:

```
1 CREATE OR REPLACE FUNCTION log_orders_changes() RETURNS TRIGGER AS $$
2 BEGIN
3     IF TG_OP = 'INSERT' THEN
4         INSERT INTO orders_log (order_id, user_id, operation, new_data)
5         VALUES (NEW.id, NEW.user_id, 'INSERT', row_to_json(NEW)::jsonb);
6         RETURN NEW;
7     ELSEIF TG_OP = 'UPDATE' THEN
8         INSERT INTO orders_log (order_id, user_id, operation, old_data, new_data)
9         VALUES (NEW.id, NEW.user_id, 'UPDATE', row_to_json(OLD)::jsonb, row_to_json(NEW)::jsonb);
10        RETURN NEW;
11    ELSEIF TG_OP = 'DELETE' THEN
12        INSERT INTO orders_log (order_id, user_id, operation, old_data)
13        VALUES (OLD.id, OLD.user_id, 'DELETE', row_to_json(OLD)::jsonb);
14        RETURN OLD;
15    END IF;
16 END;
17 $$ LANGUAGE plpgsql;
```

Below the query editor, the 'Messages' tab is selected, showing the output: 'CREATE FUNCTION' and 'Query returned successfully in 44 msec.'

3. Прив'язати тригер до відповідної таблиці, щоб автоматично фіксувати кожну зміну, покращуючи аудит змін:

The screenshot shows the pgAdmin 4 interface. The top bar includes the menu (File, Object, Tools, Edit, View, Window, Help) and the connection name 'online_store/postgres@PostgreSQL 17'. Below the toolbar, the 'Query' tab is active, displaying a SQL script to create a trigger. The script is as follows:

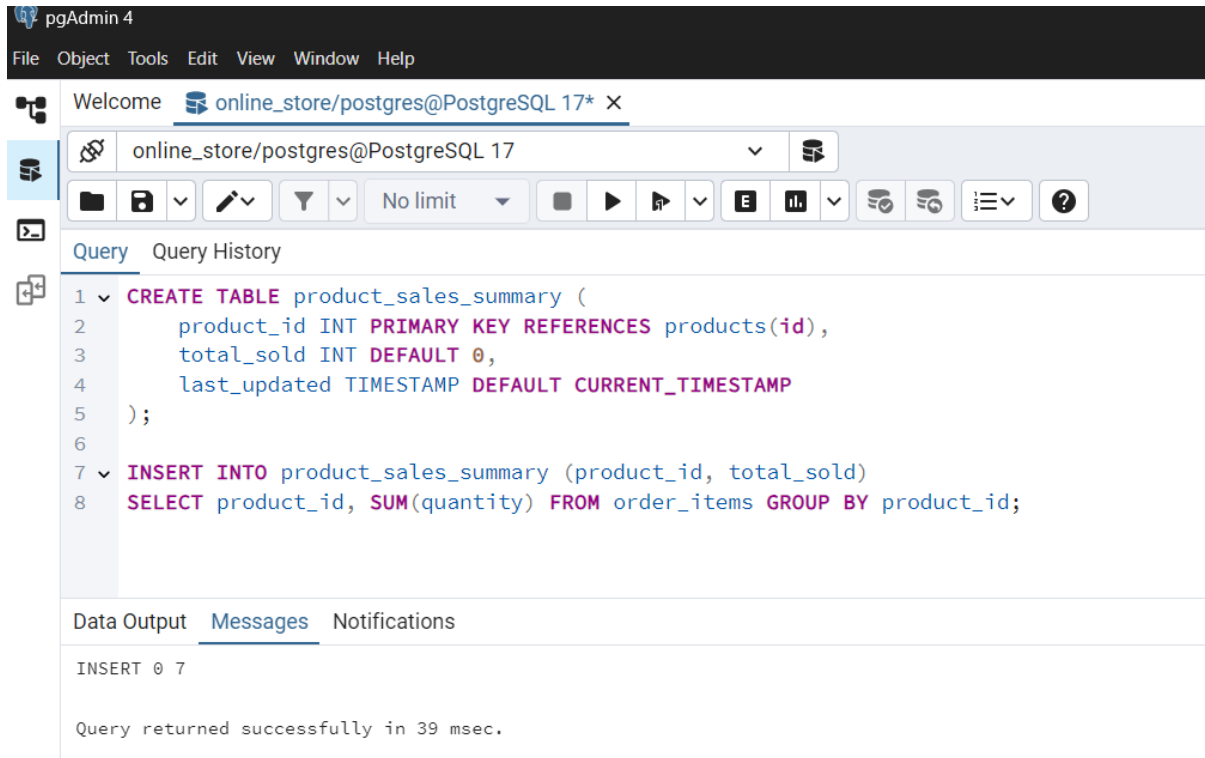
```
1 CREATE TRIGGER track_orders_changes
2 AFTER INSERT OR UPDATE OR DELETE ON orders_table
3 FOR EACH ROW
4 EXECUTE FUNCTION log_orders_changes();
```

Below the query editor, the 'Messages' tab is selected, showing the output: 'CREATE TRIGGER' and 'Query returned successfully in 51 msec.'

4. Створити тригер для автоматичного оновлення пов'язаних таблиць або заповнення значень у залежних таблицях, щоб мінімізувати дублювання даних та покращити зв'язність структури бази.

5. Використати тригер для обчислення загальних підсумків або створення автоматичних звітів на основі змін у таблицях.

Зведення продажів за товарами з підрахунком загальної кількості:



Автоматичне оновлення підсумків продажів при кожному додаванні або видаленні товару в замовленні, щоб уникнути повторного агрегування всіх даних:

pgAdmin 4

File Object Tools Edit View Window Help

Welcome online_store/postgres@PostgreSQL 17* X

online_store/postgres@PostgreSQL 17

No limit

Query Query History

```
1 CREATE OR REPLACE FUNCTION update_product_sales_summary() RETURNS TRIGGER AS $$
2 BEGIN
3     IF TG_OP = 'INSERT' THEN
4         INSERT INTO product_sales_summary (product_id, total_sold, last_updated)
5         VALUES (NEW.product_id, NEW.quantity, CURRENT_TIMESTAMP)
6         ON CONFLICT (product_id)
7         DO UPDATE SET
8             total_sold = product_sales_summary.total_sold + NEW.quantity,
9             last_updated = CURRENT_TIMESTAMP;
10        RETURN NEW;
11
12    ELSIF TG_OP = 'DELETE' THEN
13        UPDATE product_sales_summary
14        SET total_sold = total_sold - OLD.quantity,
15            last_updated = CURRENT_TIMESTAMP
16        WHERE product_id = OLD.product_id;
17        RETURN OLD;
18    END IF;
19 END;
20 $$ LANGUAGE plpgsql;
21
22 CREATE TRIGGER sync_product_sales
23 AFTER INSERT OR DELETE ON order_items
24 FOR EACH ROW
25 EXECUTE FUNCTION update_product_sales_summary();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 39 msec.

Тест логуювання INSERT:

pgAdmin 4

File Object Tools Edit View Window Help

Welcome online_store/postgres@PostgreSQL 17* X

online_store/postgres@PostgreSQL 17

No limit

Query Query History

```
1 INSERT INTO orders_table (user_id, order_date) VALUES (1, CURRENT_DATE);
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 39 msec.

Тест логування UPDATE:

The screenshot shows the pgAdmin 4 web interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main toolbar contains various icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL query:

```
1 UPDATE orders_table SET order_date = CURRENT_DATE WHERE id = 1;
```

Below the query editor, the 'Messages' tab is selected, showing the execution results:

```
UPDATE 1  
  
Query returned successfully in 46 msec.
```

Перевірка логів:

The screenshot shows the pgAdmin 4 web interface with the 'Query' tab active. The SQL query displayed is:

```
1 SELECT * FROM orders_log ORDER BY changed_at DESC;
```

The 'Data Output' tab is selected, showing the results of the query. The table has the following columns: log_id, order_id, user_id, operation, old_data, new_data, and changed_at. The results are as follows:

log_id	order_id	user_id	operation	old_data	new_data	changed_at
1	2	1	UPDATE	{"id": 1, "user_id": 1, "order_date": "2026-02-01"}	{"id": 1, "user_id": 1, "order_date": "2026-04-22"}	2026-04-22 13:45:23.057866
2	1	26	INSERT	[null]	{"id": 26, "user_id": 1, "order_date": "2026-04-22"}	2026-04-22 13:44:21.112984

Перевірка підсумків продажів:

pgAdmin 4

File Object Tools Edit View Window Help

Welcome online_store/postgres@PostgreSQL 17* x

online_store/postgres@PostgreSQL 17

No limit

Query Query History

```

1 SELECT p.name, pss.total_sold, pss.last_updated
2 FROM product_sales_summary pss
3 JOIN products p ON p.id = pss.product_id
4 ORDER BY pss.total_sold DESC;

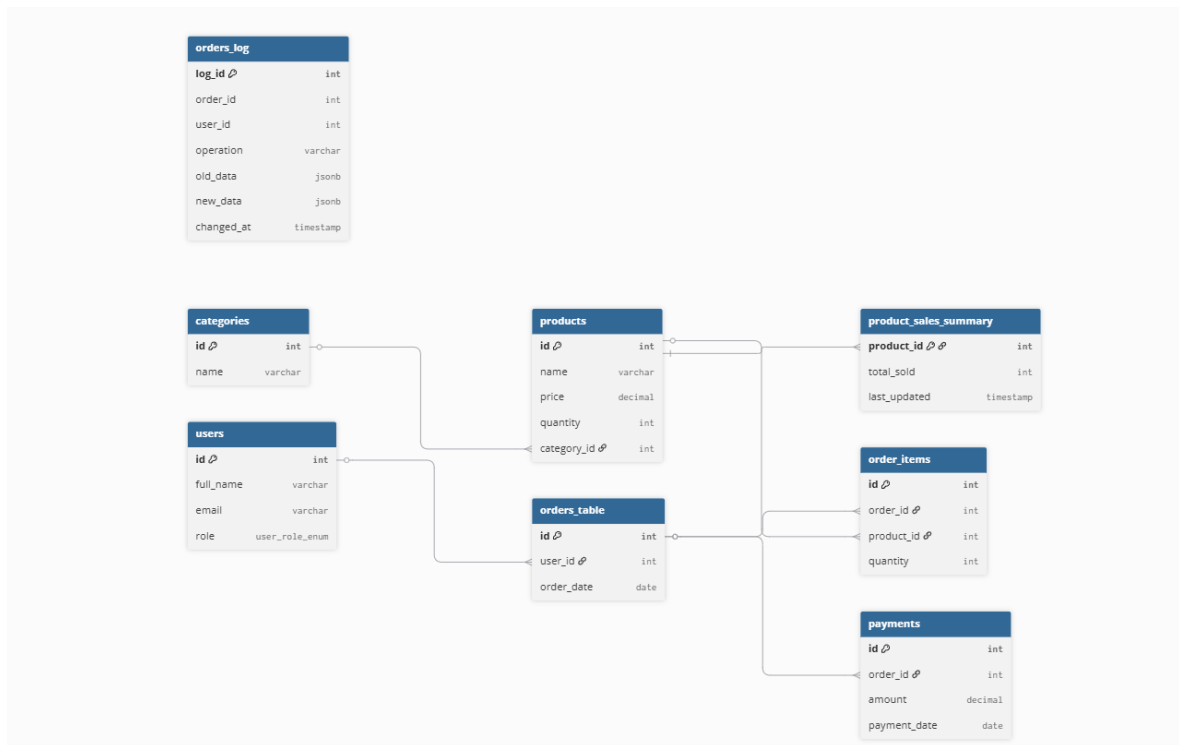
```

Data Output Messages Notifications

	name character varying (100)	total_sold integer	last_updated timestamp without time zone
1	Смарт-годинник Apple Watch	4	2026-04-22 13:37:44.181337
2	Футболка Adidas	2	2026-04-22 13:37:44.181337
3	Навушники Sony	2	2026-04-22 13:37:44.181337
4	Смартфон Samsung	1	2026-04-22 13:37:44.181337
5	Книга "Python для початківців"	1	2026-04-22 13:37:44.181337
6	Книга "Програмування на Ja...	1	2026-04-22 13:37:44.181337
7	Ноутбук Asus	1	2026-04-22 13:37:44.181337

4. Оновлення діаграми бази даних

1. Внести зміни у структуру бази даних на основі виконаних завдань, додаючи нові сутності або зв'язки між таблицями.
2. Оновити ER-діаграму (Entity-Relationship Diagram), додавши нові таблиці, стовпці, зв'язки, тригери та функції.
3. Візуалізувати нову структуру для кращого розуміння взаємодії між сутностями.
4. Переконаватися, що всі зміни документовані та відповідають логіці проєкту, забезпечуючи майбутню розширюваність бази даних.



5. Перевірка роботи

Виконала аналіз продуктивності:

pgAdmin 4

File Object Tools Edit View Window Help

Welcome online_store/postgres@PostgreSQL 17* X

online_store/postgres@PostgreSQL 17

No limit

Query Query History

```

1 EXPLAIN ANALYZE
2 SELECT COUNT(o.id), SUM(p.amount), AVG(p.amount)
3 FROM orders_table o
4 LEFT JOIN payments p ON p.order_id = o.id
5 WHERE o.user_id = 1;

```

Data Output Messages Notifications

QUERY PLAN

text

1	Aggregate (cost=31.99..32.00 rows=1 width=72) (actual time=0.074..0.075 rows=1 loops=1)
2	-> Hash Right Join (cost=3.56..31.95 rows=7 width=20) (actual time=0.053..0.062 rows=3 loops=1)
3	Hash Cond: (p.order_id = o.id)
4	-> Seq Scan on payments p (cost=0.00..24.50 rows=1450 width=20) (actual time=0.009..0.011 rows=...
5	-> Hash (cost=3.55..3.55 rows=1 width=4) (actual time=0.035..0.035 rows=3 loops=1)
6	Buckets: 1024 Batches: 1 Memory Usage: 9kB
7	-> Seq Scan on orders_table o (cost=0.00..3.55 rows=1 width=4) (actual time=0.028..0.029 rows=...
8	Filter: (user_id = 1)
9	Rows Removed by Filter: 3
10	Planning Time: 0.210 ms
11	Execution Time: 0.130 ms

pgAdmin 4

File Object Tools Edit View Window Help

Welcome online_store/postgres@PostgreSQL 17* X

online_store/postgres@PostgreSQL 17

No limit

Query Query History

```

1 EXPLAIN ANALYZE
2 SELECT * FROM get_user_order_stats(1);

```

Data Output Messages Notifications

QUERY PLAN

text

1	Function Scan on get_user_order_stats (cost=0.25..10.25 rows=1000 width=68) (actual time=0.472..0.472 rows=1 loops=...
2	Planning Time: 0.038 ms
3	Execution Time: 0.490 ms

Порівняння двох підходів показало, що використання функції `get_user_order_stats` є ефективнішим за прямий SQL-запит.

Щодо швидкості виконання: прямий запит виконувався 0.914 ms, тоді як функція — лише 0.528 ms. Це означає, що функція працює майже вдвічі швидше.